
Transformer Policies for Neural Combinatorial Optimization

Alessandro Barro
Polytechnic of Milan
Milan (MI) 20133, Italy
alessandro1.barro@mail.polimi.it

Abstract

In this work, we discuss offline reinforcement learning as a framework upon which sequential models such as Transformers can be framed to tackle combinatorial optimization problems. Taking the travelling salesman problem as a case of study, we empirically show how trajectory stitching can be leveraged by the agent to learn optimal or improved algorithms from a dataset of approximated solutions. This serves as an extension of supervised sequence-to-sequence, incorporating returns-to-go in the sequence representation instead of classic rewards. The decision policy is therefore conditioned on future returns, yielding to more informed solutions in the combinatorial space. We conclude with an experimental pipeline and result discussion.

1 Introduction

1.1 Problem Definition and Classic Solvers

Combinatorial optimization is one of the fundamentals when it comes to theoretical computer science. Problems in this discipline are hard to solve, making exact solutions computationally infeasible for large instances. Consequently, research has focused on developing efficient algorithms to obtain high-quality approximated solutions. One perfect example is the travelling salesman problem. Given a set n of cities, it consists in finding the shortest possible route that visits every city exactly once and returns to the starting point.

In one of its geometric variants, namely the symmetric euclidean TSP in 2 dimensions, finding an optimal solution for the TSP is NP-hard [17] and, among exact solvers [6, 8], the most efficient one has a time complexity of $\Theta(n^2 2^n)$. Therefore, many handcrafted solutions have been proposed to approximate the TSP, with some of them empirically reaching optimality gaps in the range of 0.1% to 0.5% on large benchmark instances (e.g. TSPLIB ones) [9]. A detailed discussion of these methods lies beyond the scope of this work; for a comprehensive overview, we refer the reader to one of the most renowned studies on the TSP [1].

1.2 Overview of Neural Solvers

Recent advances in deep learning have alimented curiosity towards learned combinatorial algorithmic strategies instead of handcrafted ones, to serve both as competitive approximations and for their potential to generalize across problem formulations.

Pointer Networks [22] introduced supervised learning to geometric problems using LSTMs with attention to model output distributions over input dependent dictionaries with variable size. For the TSP, Ptr-Nets achieved promising results on small instances, with an optimality gap of about 1.31% for 20 cities. In one training instance, it was notably able to outperform its greedy teacher.

Subsequent work shifted toward reinforcement learning, applying policy gradients to optimize Ptr-Nets directly on the tour length objective [2], showing a 0.95% gap for TSP50. This approach was later refined with the introduction of attention graph neural networks, which are trained with a greedy rollout baseline for advantage estimation [13]. An even lower gap of approximately 0.52% was achieved.

Finally, Transformer architectures have been adapted to the TSP [3, 11, 16]. The first approach [3] adopts an encoder-decoder architecture and is trained using reinforcement learning. Beam search decoding is employed during inference, and the overall method achieves a sub-percent optimality gap of approximately 0.04% on TSP50. The model in [16] is similar in spirit to our approach, yet it employs a different action selection mechanism and training objective¹.

Furthermore, our focus is to discuss a framework upon which combinatorial optimization can be framed: offline reinforcement learning [15], and its potential to extend supervised learning. In fact, despite direct on-policy objective optimization has demonstrated proficiency in solving problems like the TSP, it can still be regarded as a standalone algorithm, thereby with its own inherent bottlenecks.

2 Proposed Approach

2.1 The Travelling Salesman Problem as a Decision Process

Let $x = \{x_i\}_{i=1}^n$ denote a problem instance of n cities where each $x_i \in \mathbb{R}^2$. An admissible solution (or tour) is a permutation of the set of indices $\{1, \dots, n\}$, which we denote with $\pi_{1:n} = (\pi(1), \dots, \pi(n))$. The cost associates to each $\pi_{1:t}$ a numerical value and for $t = n$ it is given by

$$L(\pi) = \|x_{\pi(n)} - x_{\pi(1)}\|_2 + \sum_{i=1}^{n-1} \|x_{\pi(i)} - x_{\pi(i+1)}\|_2 \quad (1)$$

where $\|\cdot\|_2$ represents the euclidean norm and $\pi = \pi_{1:n}$. This setting is devisable as a finite horizon Markov decision process $(\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R})$, where the transition model $\mathcal{T}$ is known. More specifically:

- states $s \in \mathcal{S}(x)$ encode both the instance x and, for each step $t = 1, \dots, n$, the partial tour $\pi_{1:t}$. In practice, we prepare them as neural embeddings $f_\psi(x, \pi_{1:t})$;
- actions $a \in \mathcal{A}(s_t)$ indicate the unvisited cities, which are derivable from a given state s_t and the subsequently described transition dynamics $\mathcal{T}$;
- the transition dynamics $\mathcal{T}$ defines a mapping between s_t and its corresponding admissible action space $\mathcal{A}(s_t)$, thereby also determining the relative rewards;
- rewards $\mathcal{R}(s_t, a_t) = r_t$ are defined as $r_t = -\|x_{\pi(t)} - x_{a_t}\|_2$, as the euclidean variant of the TSP requires (in general, they are associated to node relationships).

A notion we will make extensive use of is the one of trajectories $\tau_{t:t+K}$: collections of (s_t, a_t, r_t) sequences between the specified time steps t and $t + K$. In the optics of reinforcement learning, the optimization objective can equivalently be expressed as learning a policy that maximizes the expected cumulative utility $G(\tau) = \sum_{t=1}^n r_t$, where $\tau = \tau_{1:n}$ denotes a complete trajectory [20].

2.2 Learning Decision Making from Offline Data

It has become clear that a combinatorial instance can be conceived as a decision making or sequential problem. Additionally we can observe that the sequential problem can, in turn, be viewed as a translation task, where x serves as the input language and the corresponding τ as the output one. Hence, the sequence-to-sequence [19] family of models emerge as a natural choice, with the Transformer [21] being the preferred architecture.

These models excel in supervised domains such as NLP and computer vision, where they

¹The model [16] learns a deterministic mapping between transitions and geometrical action representation, later to be matched with the nearest datapoint. Our approach is modeled after that of Ptr-Nets, learning a conditional probability over a discrete sequence of cities.

transform datasets into powerful pattern recognizers. To extend these gains to decision making, an area where early successes have been reported [22], we adopt offline reinforcement learning [15] rather than immediately posing an imitation learning task. In offline RL the agent receives a fixed dataset

$$\mathcal{D}_\beta = \{(\tau^{(l)}, x^{(l)})\}_{l=1}^D \quad (2)$$

where each trajectory $\tau^{(l)}$ is produced beforehand by a (suboptimal) behaviour policy $\beta(\tau)$. Our aim is to enable trajectory stitching: the agent’s ability to synthesize improved policies by recombining high-quality segments drawn from different trajectories [7]. The mechanism and its implications are examined in detail later.

Online policy gradient methods must repeatedly sample fresh instances, explore expansive action spaces, and backpropagate high-variance returns, expenses that escalate rapidly with instance size. Offline RL circumvents these costs by replaying a fixed cache of strong demonstrations: the Transformer can train indefinitely on this buffer, dedicating capacity to modeling and stitching instead of costly exploration. The tradeoff is distribution shift [15]: at test time the learned policy may visit states outside the support of $\beta(\tau)$, where its value estimates deteriorate.

A practical countermeasure is to engineer the behaviour policy $\beta(\tau)$. Each heuristic used to generate the D training examples has its own optimality gap and consistency profile. Christofides’ algorithm, for example, stays within roughly 10% of the optimum, whereas a greedy heuristic can deviate by more than 30% depending on the starting city and geometry [12]. By mixing such generators we obtain a $\beta(\tau)$ that covers the combinatorial space broadly while concentrating density in the quality regime most conducive to stitching [4, 14].

2.3 The Decision Transformer Policy

In Transformer approaches to offline RL [5, 10], trajectory representations are modeled autoregressively, introducing minimal modifications to the original Transformer architecture. In our case, such representations leverage returns-to-go $\hat{R}_t = \sum_{t'=1}^n r_{t'}$ rather than individual rewards r_t , to explicitly condition the model on future returns. Specifically, the action selection process is conditioned on K complete transitions $(s_j, a_j, \hat{R}_j)_{t-K+1:t}$.

$$\tau = (s_1, a_1, \hat{R}_1, s_2, \dots, a_{T-1}, \hat{R}_{T-1}, s_T) \quad (3)$$

Much like in the original DT work, we make use of the GPT architecture [18], which employs a decoder-only Transformer model with a causal self-attention mask. This restricts each token’s prediction to rely solely on previously generated tokens, ensuring an autoregressive generation process. Therefore, the trajectory distribution $P(\tau)$ can formally be expressed as

$$P(\tau) = \prod_{t=1}^n P_\theta(a_t \mid s_{t-K+1}, a_{t-K+1}, \hat{R}_{t-K+1}, \dots, a_{t-1}, \hat{R}_{t-1}, s_t) \quad (4)$$

with θ being the neural network’s parameters. The policy $P_\theta(a_t \mid \tau_{t-K+1:t-1}, s_t)$ is obtained by applying a softmax operation over the projection $h_t W^a$, where $h_t \in \mathbb{R}^d$ is the generated embedding corresponding to the action to choose, and $W^a \in \mathbb{R}^{n \times d}$ is the action embedding matrix. The model outputs a probability distribution over the admissible set of city indices a , and is trained to maximize the likelihood of correct action sequences using a cross entropy loss.

2.4 Planning with Beam Search Decoding

At inference time we adopt the beam search procedure to sample tours from $P(\tau)$ for a new instance x [10]. Starting from the empty trajectory, we recursively expand the B most promising partial solutions. At step t each beam element $\tau_{1:t-1}$ is extended with every admissible city $a_t \in \mathcal{A}(s_t)$, producing candidates $\tau_{1:t}$. Every candidate is scored with J , the cumulative log-probability of the trajectory.

$$J(\tau_{1:t}) = \log P(\tau_{1:t}) = \sum_{j=1}^t \log P_\theta(a_j \mid \tau_{j-K+1:j-1}, s_j) \quad (5)$$

where the conditioning context matches the one used during training. After scoring, the top B candidates become the beam for the next step; the process halts once all beams reach length n , and the best scoring complete tour is returned.

Decoding is biased with a desired return $\hat{R}_1 \in [-L^\beta, -L^*]$, an aspirational negative tour length between the teacher’s cost $-L^\beta$ and the unknown optimum $-L^*$. This simple signal discourages rote imitation and prompts the policy to seek shorter routes. The model then performs trajectory stitching: policy factorization allows splicing together the highest probability prefixes from different demonstrations, yielding improved tour generation (compared to the provided teacher examples).

3 Experimental Results and Discussion

To assess the effectiveness of our approach, we compare several algorithms on TSP instances of size 10, 20, and 50. We distinguish two cases: paper-reported results (marked with “*”) and self-conducted experiments. In this last case, we test the algorithms on 1000 test runs was generated from uniform sampling in the unit square $[0, 1]^2$. We trained a GPT architecture with 8 decoding layers, 8 attention heads, an embedding size of 256 and a total of 6.32M parameters for all the instance types. The learning rate was set to 10^{-4} and no particular fine-tuning routine was conducted. We didn’t observe overfitting and results are well-interpretable.

As for the datasets $\mathcal{D}_\beta$, we employed two different teachers: β_{greedy} and β_{mixed} . The first one serves the scope of empirical support to our discussion: it allows to discern whether the learned policy P_{greedy} can exhibit tangible improvement from β_{greedy} , a cheap teacher, thanks to trajectory stitching. Then, with β_{mixed} , we try to elevate the heuristic’s performance by researching a dataset composition that favors stitching, more than the baseline greedy-only buffer. This is done by sampling trajectories from a wider portfolio of algorithms that should provide the best possible optimum density.

Table 1: Average results on TSP10, TSP20, and TSP50

Algorithm	TSP10		TSP20		TSP50	
	Average cost	Gap (%)	Average cost	Gap (%)	Average cost	Gap (%)
greedy	3.1703	10.36	4.5150	17.84	7.0007	22.93
Christofides	3.0391	5.78	4.1582	8.53	6.3351	11.24
2-opt	2.8905	0.61	3.9264	2.48	5.9780	4.97
LKH-3	2.8728	0.00	3.8315	0.00	5.6949	0.00
Held-Karp (optimal)	2.8728	0.00	3.8315	0.00	5.6949	0.00
*Vinyals et al. [22]	2.8800	0.35	3.8800	1.31	7.6600	NA
*Bello et al. [2]	NA	NA	3.8900	1.42	5.9500	4.46
*Kool et al. [13]	NA	NA	3.8500	0.34	5.8000	1.76
*Bresson et al. [3]	NA	NA	NA	NA	5.7070	0.31
Ours (greedy)	2.9744	3.54	4.1884	9.31	???	???
Ours (mixed)	???	???	???	???	???	???

First of all, we see that the model trained with β_{greedy} not only outperforms its teacher in all the instances (relative gaps -6.18% , -7.23% , for $n = 10, 20$ and 50 respectively), but also learns an algorithm that competes with Christofides (even surpassing it in TSP10 with delta optimal gap 2.24%). To be completed...

4 Conclusion

In this paper, we used the Decision Transformer framework to address the TSP. Besides from experimental results, the main contribution lies in the extension of the supervised sequence-to-sequence in combinatorial optimization, introduced by Ptr-Nets and improved with return-to-go conditioning. The latter also prompts to a systematic way to control the cloning attitude of the model during inference,

including the induction of trajectory stitching. Nevertheless, subsequent research should describe more in detail the impact and fine-tuning of $\hat{R}_1$ on tour generation as well as "pushing the model further" with higher parameter numbers, hyperparameter fine-tuning. It may also be interesting to explore how the model performs on larger TSP instances and in different combinatorial problems.

References

- [1] David Applegate, Robert Bixby, Vašek Chvátal, and William Cook. The traveling salesman problem: A computational study. *The Traveling Salesman Problem: A Computational Study*, 01 2006.
- [2] Irwan Bello, Hieu Pham, Quoc V. Le, Mohammad Norouzi, and Samy Bengio. Neural combinatorial optimization with reinforcement learning. *CoRR*, abs/1611.09940, 2016.
- [3] Xavier Bresson and Thomas Laurent. The transformer network for the traveling salesman problem. *CoRR*, abs/2103.03012, 2021.
- [4] Jinglin Chen and Nan Jiang. Offline reinforcement learning under value and density-ratio realizability: The power of gaps, 2022.
- [5] Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Michael Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. Decision transformer: Reinforcement learning via sequence modeling. *CoRR*, abs/2106.01345, 2021.
- [6] G. Dantzig, R. Fulkerson, and S. Johnson. Solution of a large-scale traveling-salesman problem. *Journal of the Operations Research Society of America*, 2(4):393–410, 1954.
- [7] Justin Fu, Aviral Kumar, Ofir Nachum, George Tucker, and Sergey Levine. D4RL: datasets for deep data-driven reinforcement learning. *CoRR*, abs/2004.07219, 2020.
- [8] Michael Held and Richard M. Karp. A dynamic programming approach to sequencing problems. *Journal of the Society for Industrial and Applied Mathematics*, 10(1):196–210, 1962.
- [9] Keld Helsgaun. An effective implementation of the lin–kernighan traveling salesman heuristic. *European Journal of Operational Research*, 126(1):106–130, 2000.
- [10] Michael Janner, Qiyang Li, and Sergey Levine. Reinforcement learning as one big sequence modeling problem. *CoRR*, abs/2106.02039, 2021.
- [11] Yan Jin, Yuandong Ding, Xuanhao Pan, Kun He, Li Zhao, Tao Qin, Lei Song, and Jiang Bian. Pointerformer: Deep reinforced multi-pointer transformer for the traveling salesman problem, 2023.
- [12] David S Johnson and Lyle A McGeoch. The traveling salesman problem: a case study. *Local search in combinatorial optimization*, pages 215–310, 1997.
- [13] Wouter Kool, Herke van Hoof, and Max Welling. Attention, learn to solve routing problems!, 2019.
- [14] Aviral Kumar, Joey Hong, Anikait Singh, and Sergey Levine. When should we prefer offline reinforcement learning over behavioral cloning?, 2022.
- [15] Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *CoRR*, abs/2005.01643, 2020.
- [16] Daniel Liu. Decision transformer-based traveling salesman tour generation. 2023.
- [17] Christos H. Papadimitriou. The euclidean travelling salesman problem is np-complete. *Theoretical Computer Science*, 4(3):237–244, 1977.
- [18] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. Improving language understanding by generative pre-training. 2018.
- [19] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. Sequence to sequence learning with neural networks. *CoRR*, abs/1409.3215, 2014.
- [20] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. A Bradford Book, Cambridge, MA, USA, 2018.
- [21] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *CoRR*, abs/1706.03762, 2017.
- [22] Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. Pointer networks, 2017.